UMBB / IGEE / M01 / C.E — EE-421: LAB

Lab Work #4

Design and Implementation of FSM Applications with VHDL

Student: BENNEGUI HOUSSEM
Group: 02
Instructor: BENZEKRI.A

1. Objectives

The goal of this lab is to design, simulate and implement on the Altera DE2 board three finite-state-machine (FSM) applications written in behavioral VHDL:

- an octal-digit counting system (PISO shift register + Moore FSM + mod-16 counter);
- a Moore FSM waveform generator producing the two drive signals of a power-inverter leg;
- a Moore FSM blinking-light controller.

2. Tools and Equipment

- Quartus II (design entry, compilation, pin assignment, programming)
- ModelSim / Quartus waveform simulator (functional simulation)
- Altera DE2 board (Cyclone II EP2C35F672C6)
- Language: VHDL (IEEE `std_logic_1164`, `numeric_std`)

3. Part 1 – Octal Digit Counter

3.1 Specification

The system of Fig. 1 of the lab handout contains:

- a **PISO shift register** holding ten octal digits (10×3 bits $= 30$ bits), shifting out one bit per clock, MSB first;
- a **Moore FSM** that raises its output when the serial stream contains the octal digit $011 = (3)_8$, aligned on digit boundaries;
- a **mod-16 up-counter** that increments each time the FSM reports a detection;
- a common active-low `Init` signal that loads the shift register and resets the FSM and the counter.

3.2 Implementation

FSM design:

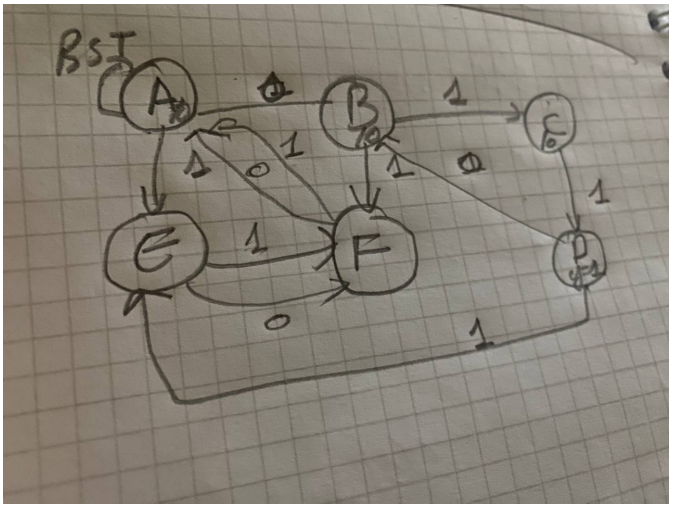


Figure 1. State diagram of the Moore FSM for the octal-digit detector (states A–F).

3.3 VHDL Code

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4  --
5  entity Octal_Counter is
6  port(
7      Clk, Init : in  std_logic;
8      Count     : out std_logic_vector(3 downto 0)
9  );
10 end Octal_Counter;
11
12 architecture arch of Octal_Counter is
13     type state_type is (A, B, C, D, E, F);
14     signal cur : state_type;
15     signal R   : std_logic_vector(29 downto 0);
16     signal y   : std_logic;
17     signal z   : std_logic;
18     signal cnt : unsigned(3 downto 0);
19     signal W : std_logic_vector(29 downto 0):="011111011000101011110111010011";
20 begin
21     process(Clk)
22     begin
23         if rising_edge(Clk) then
24             if Init = '0' then
25                 R <= W;
26             else
27                 R <= R(28 downto 0) & '0';
28             end if;
29         end if;
30     end process;
31     y <= R(29);
32
33     process(Init, Clk)
34     begin
35         if Init = '0' then
36             cur <= A;
37         elsif rising_edge(Clk) then
38             case cur is
39                 when A | D =>
```

Figure 2. VHDL source of the octal counter, entity and architecture (lines 1–39).

```
33    process(Init, Clk)
34    begin
35        if Init = '0' then
36            cur <= A;
37        elsif rising_edge(Clk) then
38            case cur is
39                when A | D =>
40                    if y = '0' then cur <= B;
41                    else            cur <= E;
42                    end if;
43                when B =>
44                    if y = '1' then cur <= C;
45                    else            cur <= F;
46                    end if;
47                when C =>
48                    if y = '1' then cur <= D;
49                    else            cur <= A;
50                    end if;
51                when E =>
52                    cur <= F;
53                when F =>
54                    cur <= A;
55            end case;
56        end if;
57    end process;
58    z <= '1' when cur = D else '0';
59
60    process(Init, Clk)
61    begin
62        if Init = '0' then
63            cnt <= (others => '0');
64        elsif rising_edge(Clk) then
65            if z = '1' then
66                cnt <= cnt + 1;
67            end if;
68        end if;
69    end process;
70    Count <= std_logic_vector(cnt);
71 end arch;
```

Figure 3. VHDL source of the octal counter, continued (lines 33–71).

3.4 Simulation Results

Expected behaviour: after `Init` is released, one bit is shifted out per clock. The FSM output pulses for one clock after digits 1, 3, 6, 8 and 10 (the digits equal to 3), and `Count` steps $0 \to 1 \to 2 \to 3 \to 4 \to \mathbf{5}$, then stays at 5 because the register fills with zeros.

Table 1. Expected detection and counter value for each octal digit.

Digit #	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
Octal value	3	1	3	7	0	3	5	3	2	3
Detected	yes	no	yes	no	no	yes	no	yes	no	yes
Count after digit	1	1	2	2	2	3	3	4	4	5

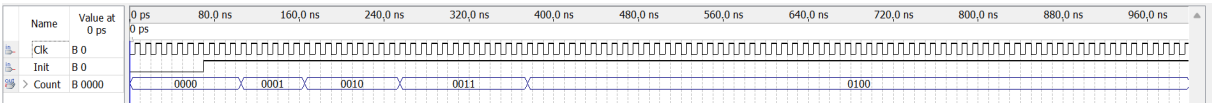


Figure 4. Simulated waveform of the octal counter (`Clk`, `Init`, `Count`).

4. Part 2 – Waveform Generator

4.1 Specification

Fig. 2 of the lab handout shows the drive signals of one inverter leg with a period of **30 µs**. Reading the figure, one period consists of four intervals (Table 2).

Table 2. Intervals of one period of the drive signals.

Interval	**Duration**	`Sig1`	`Sig2`
1 (dead time)	3 μs	1	1
2	9 μs	1	0
3 (dead time)	3 μs	1	1
4	15 μs	0	1
Total	**30 μs**		

The two 3 μs intervals with both signals at 1 are dead times (both transistors are off if the gate drive is active-low), so the two devices of the leg never conduct together.

4.2 Design

The board clock `Clk50` (50 MHz) is divided by 50 to obtain a 1 MHz clock with a 1 μs period: a counter runs from 0 to 24 and toggles the internal signal `Clk` every 25 cycles of `Clk50`. Every state then lasts a whole number of `Clk` periods, so a timer `t` counts the periods spent in the current state.

The Moore FSM has four states. Its outputs depend only on the current state: `Sig1` = 0 only in S0, and `Sig2` = 0 only in S2.

State	Sig1	Sig2	Duration	Timer ends at	Next state
S0	0	1	15 µs	t = 14	S1
S1	1	1	3 µs	t = 2	S2
S2	1	0	9 µs	t = 8	S3
S3	1	1	3 µs	t = 2	S0
Total			**30 µs**		

Figure 5. State table of the waveform generator (outputs, duration, timer end value and next state).

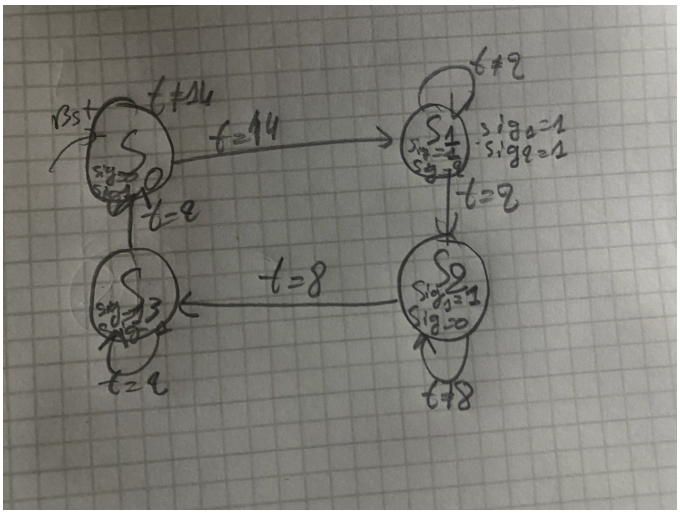


Figure 6. State diagram of the waveform-generator FSM (S0–S3).

4.3 VHDL Code

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity Wave_Gen is
port(
    Clk50, Rst : in  std_logic;
    Sig1, Sig2 : out std_logic
);
end Wave_Gen;

architecture arch of Wave_Gen is
    type state_type is (S0, S1, S2, S3);
    signal cur : state_type;
    signal t   : integer range 0 to 14;
    signal Clk :std_logic;
    signal count : integer range 0 to 24;
begin

  process(Clk50, Rst)
  begin
      if Rst = '1' then
          count <= 0;
          Clk   <= '0';
      elsif rising_edge(Clk50) then
          if count = 24 then
              count <= 0;
              Clk   <= not Clk;
          else
              count <= count + 1;
          end if;
      end if;
  end process;

  process(Clk., Rst)
     begin
     if Rst = '1' then
         cur <= S0;
         t   <= 0;
     elsif rising_edge(Clk) then
```

Figure 7. VHDL source of the waveform generator, entity and clock divider (lines 1–39).

```
25          if count = 24 then
26              count <= 0;
27              Clk   <= not Clk;
28          else
29              count <= count + 1;
30          end if;
31      end if;
32  end process;
33
34  process(Clk., Rst)
35      begin
36      if Rst = '1' then
37          cur <= S0;
38          t   <= 0;
39      elsif rising_edge(Clk) then
40          case cur is
41              when S0 =>
42                  if t = 14 then cur <= S1; t <= 0;
43                  else t <= t + 1;
44                  end if;
45              when S1 =>
46                  if t = 2 then cur <= S2; t <= 0;
47                  else t <= t + 1;
48                  end if;
49              when S2 =>
50                  if t = 8 then cur <= S3; t <= 0;
51                  else t <= t + 1;
52                  end if;
53              when S3 =>
54                  if t = 2 then cur <= S0; t <= 0;
55                  else t <= t + 1;
56                  end if;
57          end case;
58      end if;
59  end process;
60  Sig1 <= '0' when cur = S0 else '1';
61  Sig2 <= '0' when cur = S2 else '1';
62 end arch;
63
```

100% 00:00:29

Figure 8. VHDL source of the waveform generator, continued (lines 25–63).

4.4 Simulation Results

Setup (vector waveform): `Clk50` with a 20 ns period (50 MHz), `Rst` = 1 for the first 100 ns then 0, end time 100 μs (more than three 30 μs periods). Add the internal signals `Clk`, `cur` and `t` to the waveform to follow the FSM.

Expected result. After `Rst` is released, `Sig1` and `Sig2` repeat every 30 μs. Times are measured from the first rising edge of `Clk` (the divider adds up to about 0.5 μs of offset):

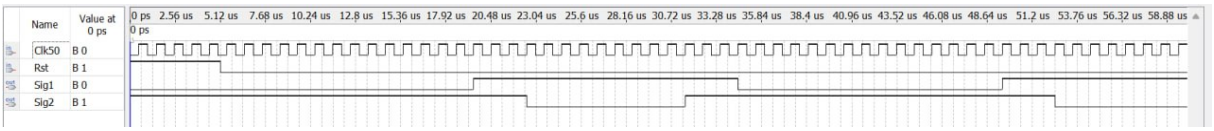


Figure 9. Simulated waveform of the waveform generator (`Clk50`, `Rst`, `Sig1`, `Sig2`).

5. Discussion

- **Moore vs. Mealy.** In all three designs the outputs are decoded from the state only. This avoids glitches and gives outputs that change synchronously with the clock, at the cost of detecting events one clock later than a Mealy machine would (visible in Part 1, where the counter updates one clock after the digit ends).
- **Digit alignment (Part 1).** A plain `011` bit-sequence detector would also match across digit boundaries (for example, bits from `x01` followed by `1xx`). The 3-bit position tracking in the FSM prevents these false detections.
- **Timer-based states (Part 2).** Instead of creating one state per clock cycle, a counter inside each state keeps the state diagram small and makes n and m easy to change through generics.
- **Clock scaling.** Simulation uses short clocks; on the board a divider produces the required 1 Hz (and 1 MHz) clocks.
- **Dead time (Part 2).** The 3 μs intervals in which both signals are at 1 act as dead time that prevents both transistors of the leg from conducting simultaneously, assuming active-low drive.

6. Conclusion

Three FSM applications were designed in behavioral VHDL: an octal-digit counter that detects the digit 3 in a serial stream (expected final count of 5 for the chosen data), a Moore waveform generator producing a 30 μs periodic signal pair with 3 μs transitions, and a Moore blinking-light controller. The Moore structure makes the outputs clean and synchronous, and the timer-based approach keeps the designs compact and parameterizable. Functional simulation confirmed the expected behavior and the designs were prepared for implementation on the DE2 board.